Name: Mimanshu Gahlaut

UID: 24BAI70038

Course: BE-CSE (AI&ML)

Subject: Database Management System

Experiment 7: Create and Analyze Views

1. Aim of the Session

To design and implement a materialised view and to compare and analyse execution time and performance differences between simple views, complex views, and materialized views, thereby understanding their impact on query optimization and system performance.

2. Software Requirements:

- Database Management System:
  - Oracle Database Express Edition (Oracle XE)
  - PostgreSQL Database

- Database Administration Tool / Client Tool:
  - Oracle SQL Developer (for Oracle XE)
  - pgAdmin (for PostgreSQL)

3. Objective of the Session

To create simple views, complex views, and materialized views, and to evaluate their performance by comparing query execution time for each, highlighting the advantages of materialized views in enterprise-level applications.

4. Practical / Experiment Steps

The work was carried out through the following activities:

1. **Database Design:** Created two related tables (`departments` and `employees`) with proper primary and foreign key constraints.
2. **Basic View Creation:** Developed a simple view (`V_BASIC`) to display employees with salaries above a certain limit.

3. **Advanced View Development:** Constructed a complex view (`V_ADVANCED`) using joins and aggregation functions like `SUM()` and `AVG()` to analyze department-wise salaries.
4. **Materialized View Setup:** Created a materialized view (`V_STORE`) to store precomputed results for faster data retrieval.
5. **Performance Evaluation:** Used query analysis tools such as `EXPLAIN ANALYZE` to compare execution time of different views.
6. **Data Synchronization:** Applied refresh operations to update the materialized view after changes in base tables.

5. Procedure of the Practical

Execution was performed in the following order:

- Connected to the PostgreSQL/Oracle database using the appropriate client tool.
- Created `departments` and `employees` tables and inserted sample data.
- Defined a simple view to filter employees based on salary conditions.
- Built a complex view using JOIN and GROUP BY to calculate department-wise salary statistics.
- Created a materialized view using the same query to store results physically.
- Performed updates on base tables and refreshed the materialized view.
- Executed `EXPLAIN ANALYZE` on all views to compare performance.
- Observed differences in execution time and efficiency.
- Documented results and conclusions.

6. I/O Analysis (Input / Output Analysis)

Input Queries

SQL

```sql
CREATE TABLE departments(
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(25)
);

CREATE TABLE employees(
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(25),
    dept_id INT REFERENCES departments(dept_id),
    salary NUMERIC
);
```

Query | Query History

```sql
CREATE TABLE departments(
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(25)
);

CREATE TABLE employees(
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(25),
    dept_id INT REFERENCES departments(dept_id),
    salary NUMERIC
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 210 msec.
```

```sql
INSERT INTO departments VALUES (1, 'IT'), (2, 'Finance'), (3, 'Marketing');

INSERT INTO employees VALUES (201, 'Rahul', 1, 90000);
INSERT INTO employees VALUES (202, 'Neha', 2, 60000);
INSERT INTO employees VALUES (203, 'Karan', 1, 85000);
INSERT INTO employees VALUES (204, 'Simran', 3, 50000);
```

```sql
INSERT INTO departments VALUES (1, 'IT'), (2, 'Finance'), (3, 'Marketing');

INSERT INTO employees VALUES (201, 'Rahul', 1, 90000);
INSERT INTO employees VALUES (202, 'Neha', 2, 60000);
INSERT INTO employees VALUES (203, 'Karan', 1, 85000);
INSERT INTO employees VALUES (204, 'Simran', 3, 50000);
```

Data Output | Messages | Notifications

```
INSERT 0 1

Query returned successfully in 120 msec.
```

```sql
--Simple View--
CREATE VIEW V_BASIC AS
SELECT emp_name, salary FROM employees WHERE salary > 70000;

SELECT * FROM V_BASIC;
```

```sql
CREATE VIEW V_BASIC AS
SELECT emp_name, salary FROM employees WHERE salary > 70000;

SELECT * FROM V_BASIC;
```

Data Output | Messages | Notifications

	emp_name character varying (25)	salary numeric
1	Rahul	90000
2	Karan	85000

```sql
--Complex View--
CREATE VIEW V_ADVANCED AS
SELECT d.dept_name, SUM(e.salary) AS total_salary, AVG(e.salary) AS avg_salary
FROM employees e JOIN departments d
ON e.dept_id = d.dept_id
GROUP BY d.dept_name;

SELECT * FROM V_ADVANCED;
```

```sql
CREATE VIEW V_ADVANCED AS
SELECT d.dept_name, SUM(e.salary) AS total_salary, AVG(e.salary) AS avg_salary
FROM employees e JOIN departments d
ON e.dept_id = d.dept_id
GROUP BY d.dept_name;

SELECT * FROM V_ADVANCED;
```

Data Output | Messages | Notifications

	dept_name character varying (25)	total_salary numeric	avg_salary numeric
1	Marketing	50000	50000.000000000000
2	Finance	60000	60000.000000000000
3	IT	175000	87500.000000000000

```sql
--Materialized View--
CREATE MATERIALIZED VIEW V_STORE AS
SELECT d.dept_name, SUM(e.salary) AS total_salary, AVG(e.salary) AS avg_salary
FROM employees e JOIN departments d
ON e.dept_id = d.dept_id
GROUP BY d.dept_name;

SELECT * FROM V_STORE;
```

```
36  CREATE MATERIALIZED VIEW V_STORE AS
37  SELECT d.dept_name, SUM(e.salary) AS total_salary, AVG(e.salary) AS avg_salary
38  FROM employees e JOIN departments d
39  ON e.dept_id = d.dept_id
40  GROUP BY d.dept_name;
41
42  SELECT * FROM V_STORE;
```

Data Output | Messages | Notifications

Showin

	dept_name character varying (25)	total_salary numeric	avg_salary numeric
1	Marketing	50000	50000.000000000000
2	Finance	60000	60000.000000000000
3	IT	175000	87500.000000000000

```
--Refresh--
REFRESH MATERIALIZED VIEW V_STORE;
```

```
44
45  REFRESH MATERIALIZED VIEW V_STORE;
```

Data Output | Messages | Notifications

```
REFRESH MATERIALIZED VIEW

Query returned successfully in 90 msec.
```

```
--Performance Check--
EXPLAIN ANALYZE SELECT * FROM V_BASIC;
```

```
47
48  EXPLAIN ANALYZE SELECT * FROM V_BASIC;
49
```

Data Output | Messages | Notifications

	QUERY PLAN text
1	Seq Scan on employees (cost=0.00..17.50 rows=200 width=100) (actual time=0.038..0.043 rows=2.00 loop...
2	Filter: (salary > '70000'::numeric)
3	Rows Removed by Filter: 2
4	Buffers: shared hit=1
5	Planning Time: 0.211 ms
6	Execution Time: 0.072 ms

```
EXPLAIN ANALYZE SELECT * FROM V_ADVANCED;
```

```
50
51  EXPLAIN ANALYZE SELECT * FROM V_ADVANCED;
```

Data Output | Messages | Notifications

Showing rows: 1 to 16

	QUERY PLAN text
1	HashAggregate (cost=48.81..51.81 rows=200 width=132) (actual time=0.130..0.138 rows=3.00 loops=1)
2	Group Key: d.dept_name
3	Batches: 1 Memory Usage: 32kB
4	Buffers: shared hit=2
5	-> Hash Join (cost=28.23..45.81 rows=600 width=100) (actual time=0.093..0.100 rows=4.00 loops=1)
6	Hash Cond: (e.dept_id = d.dept_id)
7	Buffers: shared hit=2
8	-> Seq Scan on employees e (cost=0.00..16.00 rows=600 width=36) (actual time=0.039..0.041 rows=4.00 loops=1)
9	Buffers: shared hit=1
10	-> Hash (cost=18.10..18.10 rows=810 width=72) (actual time=0.041..0.041 rows=3.00 loops=1)
11	Buckets: 1024 Batches: 1 Memory Usage: 9kB
12	Buffers: shared hit=1
13	-> Seq Scan on departments d (cost=0.00..18.10 rows=810 width=72) (actual time=0.016..0.019 rows=3.00 loo...
14	Buffers: shared hit=1
15	Planning Time: 0.439 ms
16	Execution Time: 0.236 ms

```
EXPLAIN ANALYZE SELECT * FROM V_STORE;
```

```
53
54  EXPLAIN ANALYZE SELECT * FROM V_STORE;
```

Data Output | Messages | Notifications

	QUERY PLAN text
1	Seq Scan on v_store (cost=0.00..15.10 rows=510 width=132) (actual time=0.036..0.039 rows=3.00 loop...
2	Buffers: shared hit=1
3	Planning Time: 0.130 ms
4	Execution Time: 0.068 ms

7. Learning Outcome

- Learned the difference between simple, complex, and materialized views.
- Understood how materialized views improve performance.
- Gained knowledge of query analysis using EXPLAIN ANALYZE.
- Learned how to refresh and manage stored views.